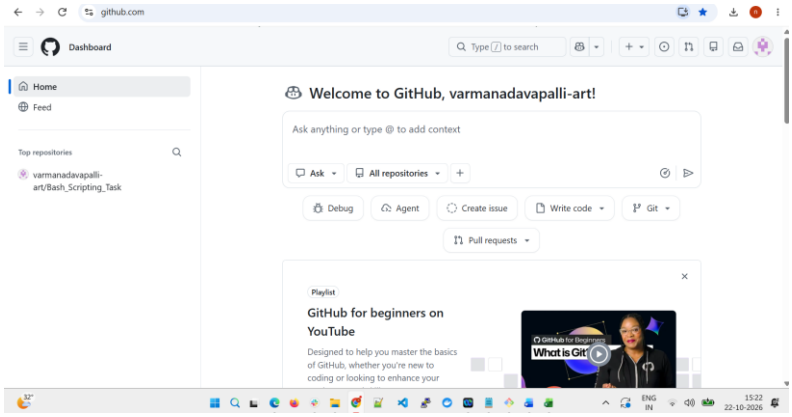
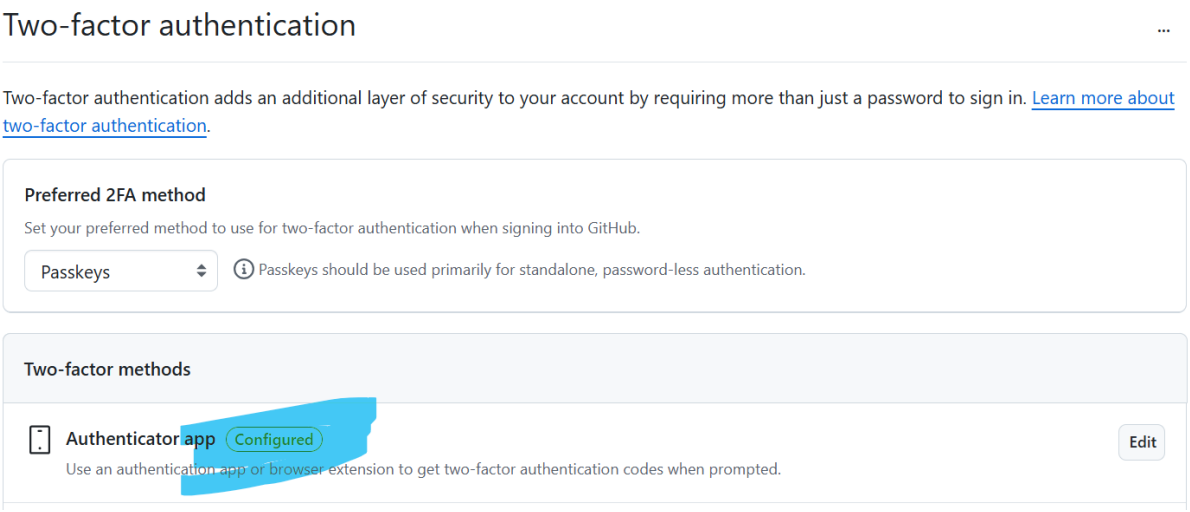
Git & GITHUB Task video-9

\\

1. Watch the video and write notes.
2. Create a GitHub account.



3. Set up MFA (Multi-Factor Authentication) for the GitHub account using a mobile app.



4. Execute all commands shown in the video.

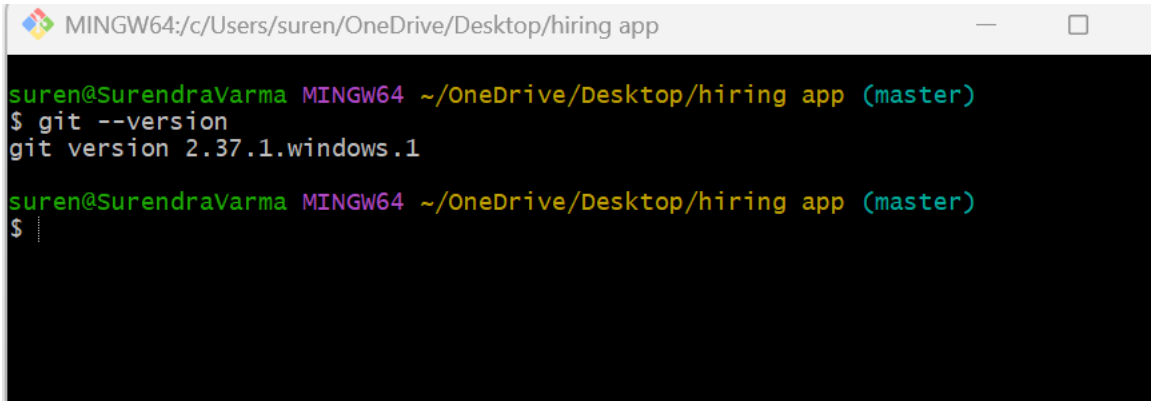
Install Git

sudo apt update

sudo apt install git -y

git --version

I used sudo apt update to update the package information. Then I used sudo apt install git -y to install Git. Finally, I used git --version to verify that Git was installed successfully.



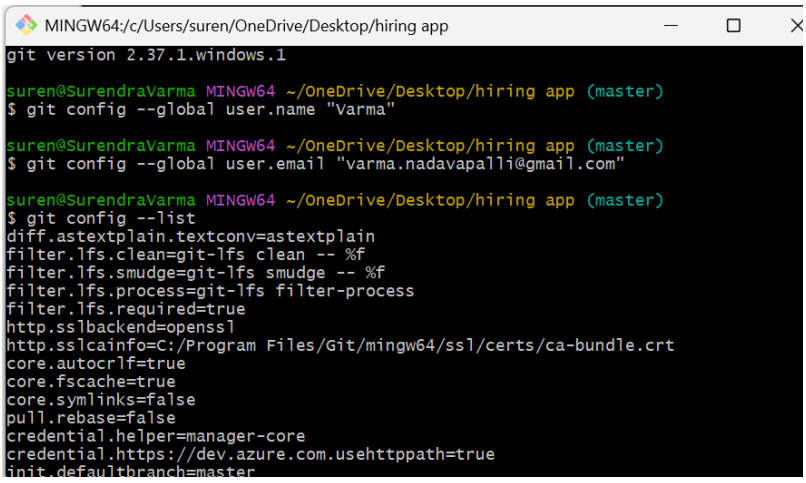
2. Configure Git

git config --global user.name "Varma"

git config --global user.email "varma.nadavapalli@gmail.com"

git config --list

I used git config --global user.name to configure my Git username. I used git config --global user.email to configure my email address. These details are associated with the commits I create. I used git config --list to verify my configuration.



3. Create a project directory

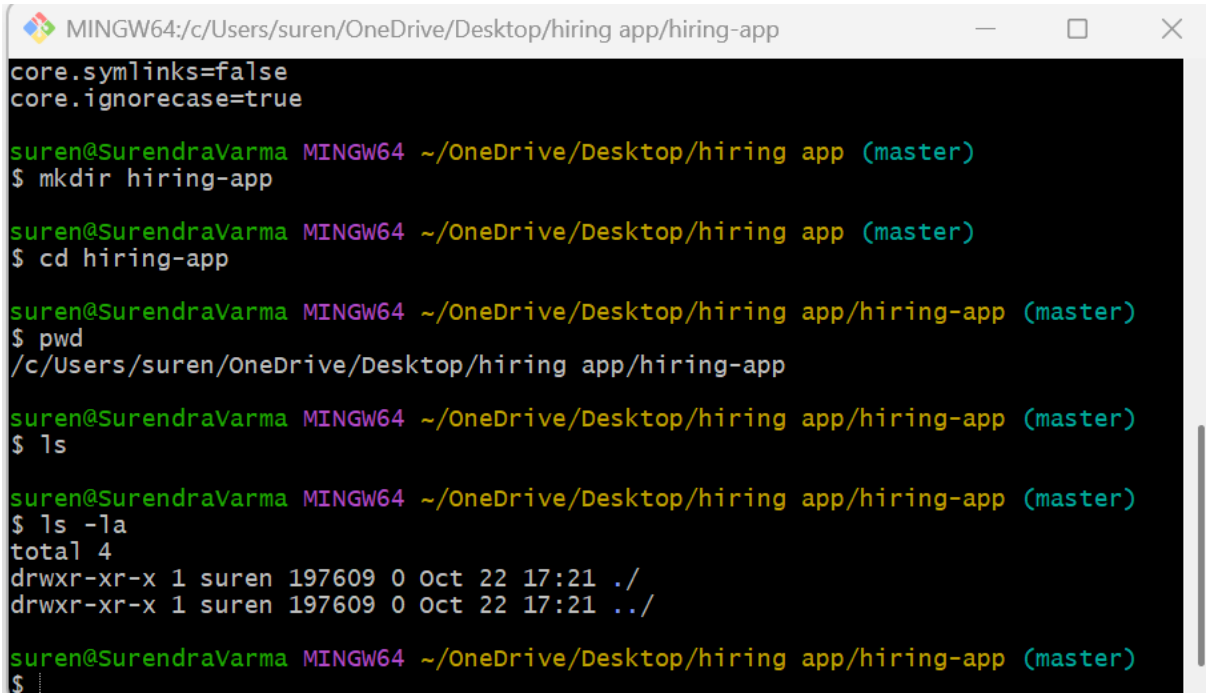
mkdir hiring-app

cd hiring-app

pwd

ls

ls -la

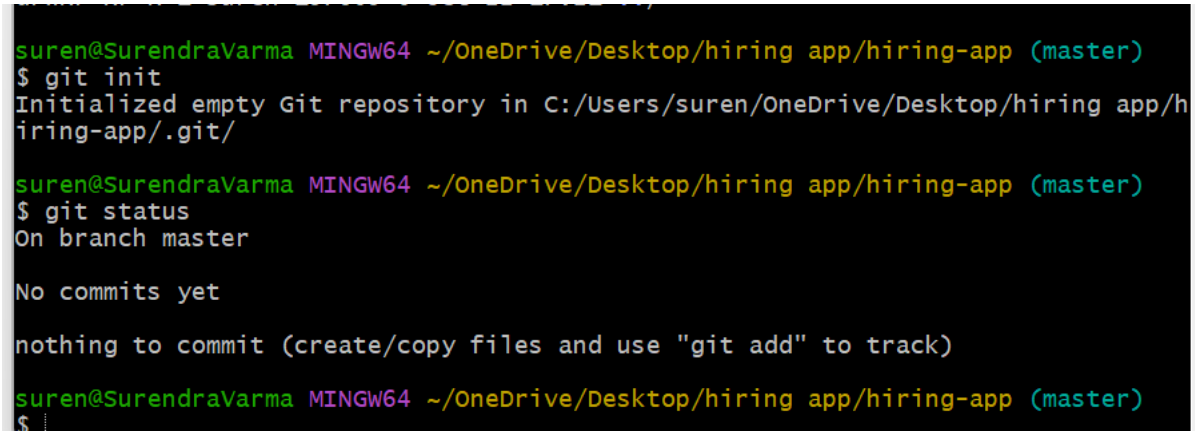


I used mkdir hiring-app to create my project directory. Then I used cd hiring-app to enter the directory. I used pwd to check my current location and ls to see the files. I used ls -la to also display hidden files.

4. Initialize Git

git init

git status

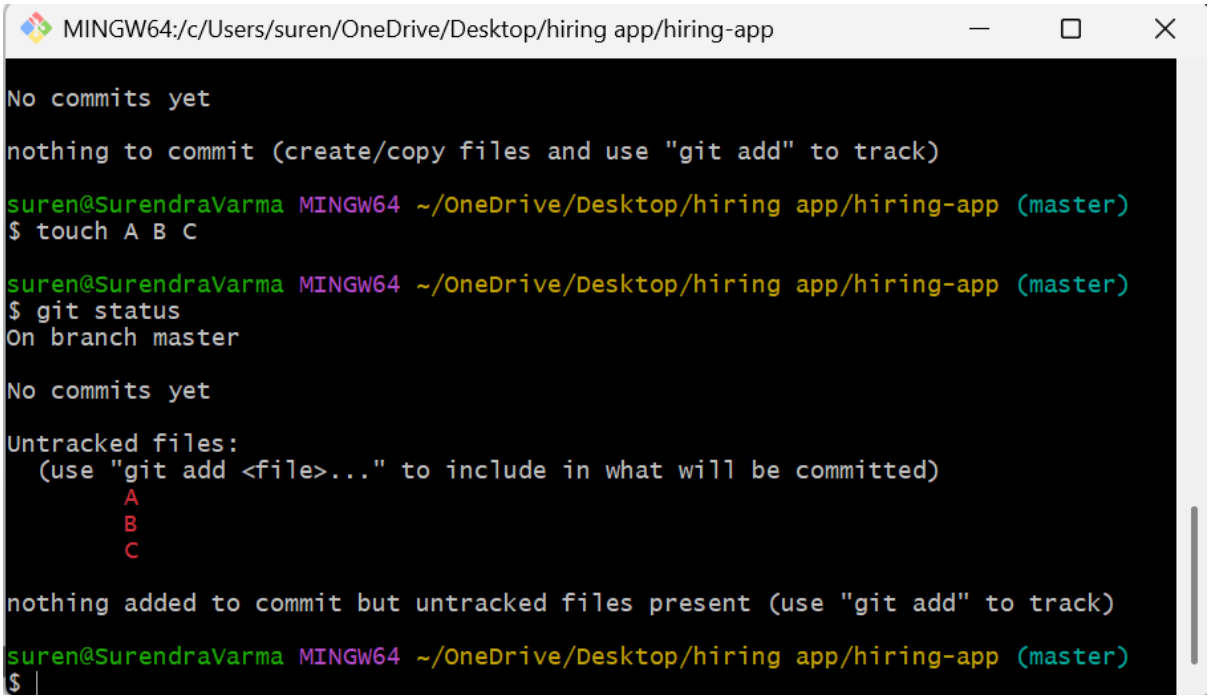


I used git init to initialize Git in my project directory. This created a hidden .git directory, which stores Git's repository information. Then I used git status to check the current status of my repository.

5. Create files

touch A B C

git status



I used touch A B C to create three files. Then I used git status to check the repository.

Git showed these files as untracked because I had not yet added them to Git's staging area

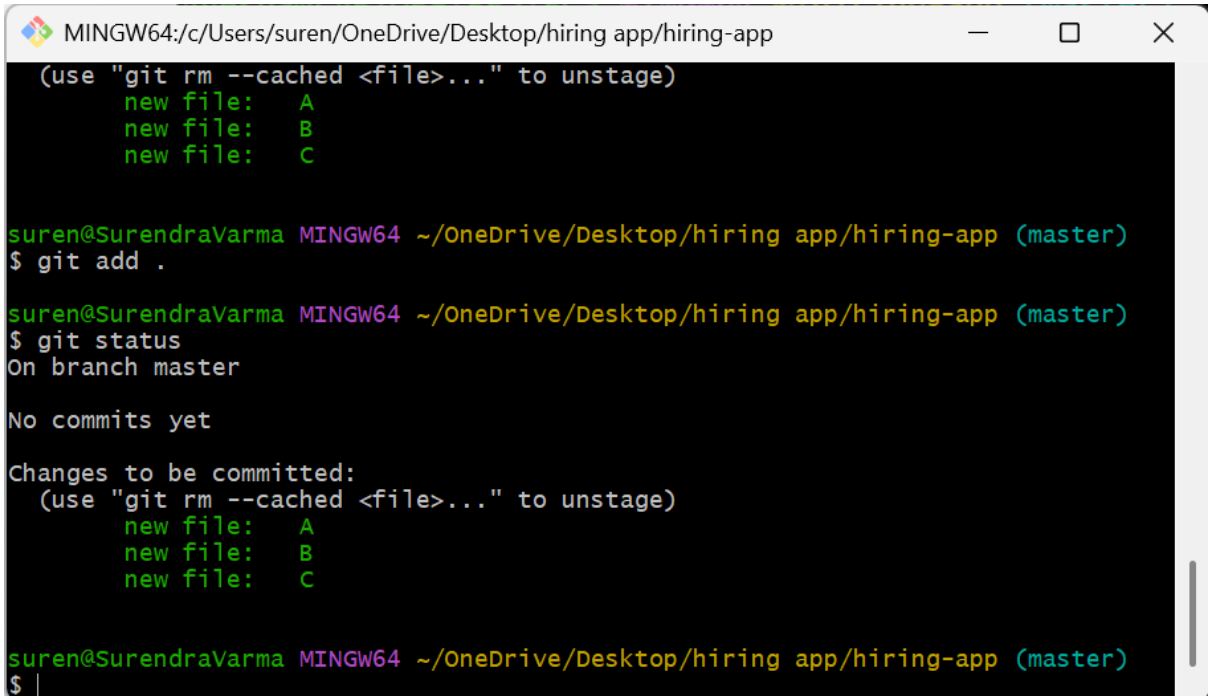
6. Add files to staging

git add A

git add A B C

git add

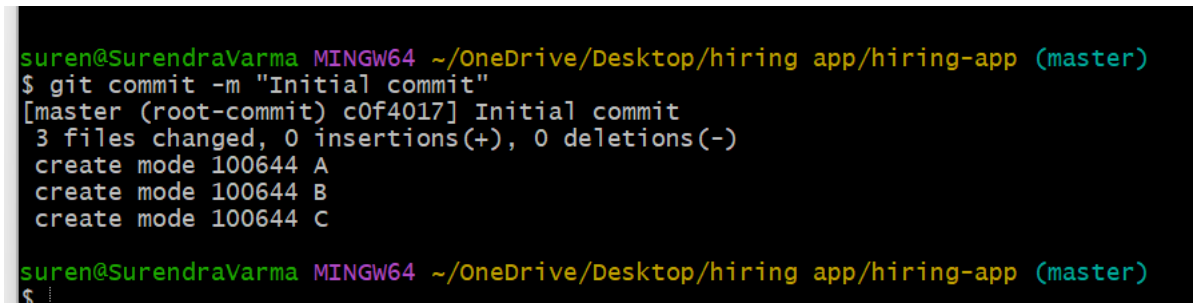
git status



I used git add to move files from the working directory to the staging area. I can add one file, multiple files, or all files using git add .. Then I used git status to verify which files were staged.

7. Commit the changes

git commit -m "Initial commit"



I used git commit to permanently save the staged changes in the Git repository. I provided "Initial commit" as the commit message so I could understand what changes were made.

8. Check commit history

git log

git log --oneline

git log -4

```
suren@SurendraVarma MINGW64 ~/OneDrive/Desktop/hiring app/hiring-app (master)
$ git log
commit c0f40171b84e3b72cf218afbb9946cec5a09daa2 (HEAD -> master)
Author: Varma <varma.nadavapalli@gmail.com>
Date:   Thu Oct 22 17:31:53 2026 +0530

    Initial commit

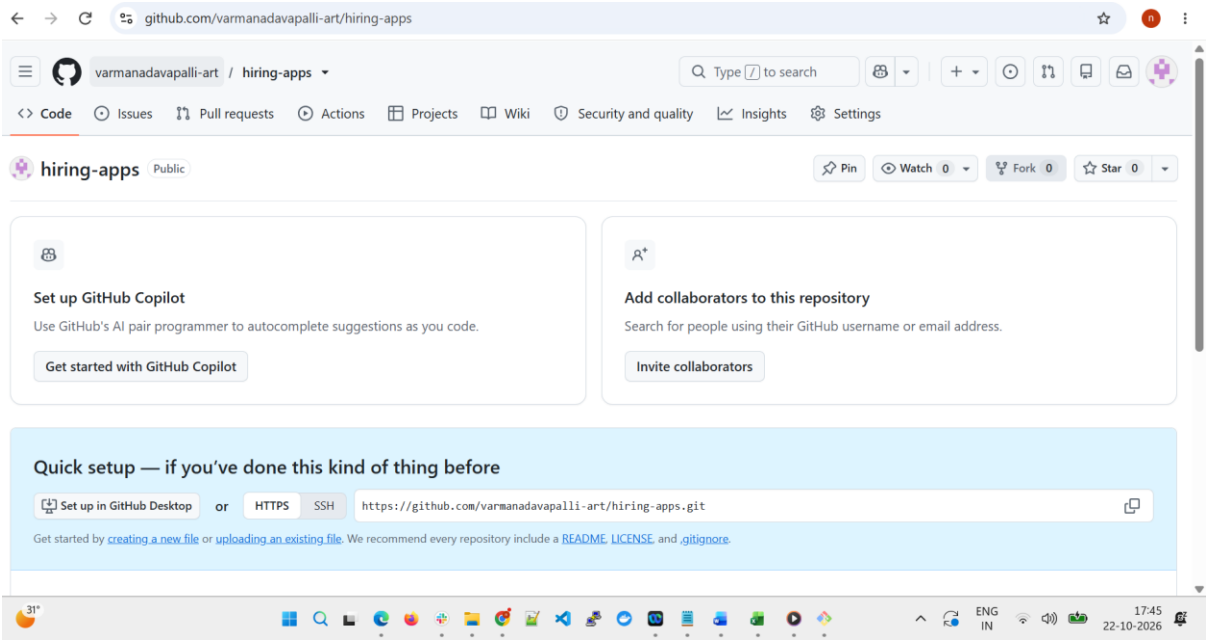
suren@SurendraVarma MINGW64 ~/OneDrive/Desktop/hiring app/hiring-app (master)
$ git log --oneline
c0f4017 (HEAD -> master) Initial commit

suren@SurendraVarma MINGW64 ~/OneDrive/Desktop/hiring app/hiring-app (master)
$ git log -4
commit c0f40171b84e3b72cf218afbb9946cec5a09daa2 (HEAD -> master)
Author: Varma <varma.nadavapalli@gmail.com>
Date:   Thu Oct 22 17:31:53 2026 +0530

    Initial commit

suren@SurendraVarma MINGW64 ~/OneDrive/Desktop/hiring app/hiring-app (master)
$
```

I used git log to view the complete commit history. I used git log --oneline to see the commits in a short format. I used git log -4 to display the latest four commits.



GitHub repo

I created a repository named hiring-app on GitHub. Then I used git remote add origin to connect my local Git repository with the GitHub repository. I used git remote -v to verify the remote URL.

```
    Initial commit

suren@SurendraVarma MINGW64 ~/OneDrive/Desktop/hiring app/hiring-app (master)
$ git remote add origin https://github.com/varmanadavapalli-art/hiring-apps.git

suren@SurendraVarma MINGW64 ~/OneDrive/Desktop/hiring app/hiring-app (master)
$ git remote
origin

suren@SurendraVarma MINGW64 ~/OneDrive/Desktop/hiring app/hiring-app (master)
$ git remote -v
origin  https://github.com/varmanadavapalli-art/hiring-apps.git (fetch)
origin  https://github.com/varmanadavapalli-art/hiring-apps.git (push)

suren@SurendraVarma MINGW64 ~/OneDrive/Desktop/hiring app/hiring-app (master)
$
```

10. Rename branch to main

git branch

git branch--show-current

git branch -M main

```
suren@SurendraVarma MINGW64 ~/OneDrive/Desktop/hiring app/hiring-app (master)
$ git remote -v
origin  https://github.com/varmanadavapalli-art/hiring-apps.git (fetch)
origin  https://github.com/varmanadavapalli-art/hiring-apps.git (push)

suren@SurendraVarma MINGW64 ~/OneDrive/Desktop/hiring app/hiring-app (master)
$ git branch
* master

suren@SurendraVarma MINGW64 ~/OneDrive/Desktop/hiring app/hiring-app (master)
$ git branch -M main

suren@SurendraVarma MINGW64 ~/OneDrive/Desktop/hiring app/hiring-app (main)
$
```

I used git branch to check my current branch. I used git branch --show-current to

display the current branch name. Then I used git branch -M main to rename my branch

to main.

11. Push code to GitHub

git push -u origin main

ssh key:

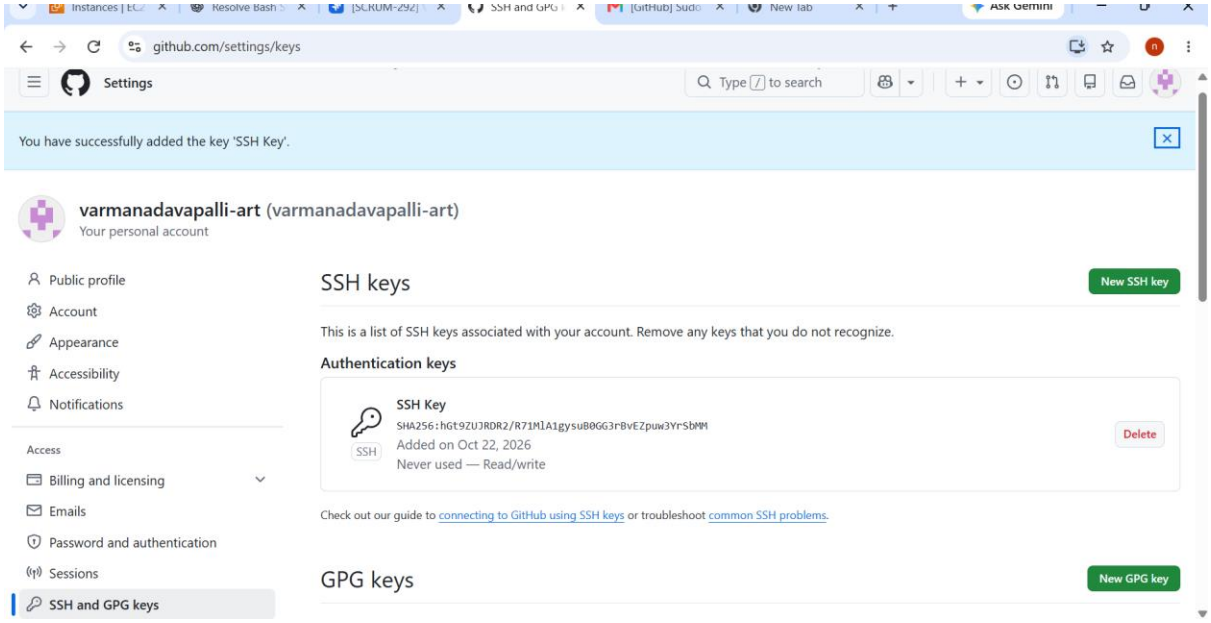
ssh-rsa
AAAAB3NzaC1yc2EAAAADAQABAAABgQCw7KrzCFbmQctVC0c173Qb89FlSqNCV3N2TSaSkIFWX4ZQU
XQ8U9Kb7vO/o48vuG5Km6xkC+J+gu/jOZhcazULAI7hzUSKS3dne7BSfE3Avz542Sc3FZXwpedFFf8Eap3

O5vGl04AV1dT4//CpQZCZi/BinqApEmDejxztHTEX0Mhi+3M0J7iJofceKAYJXyfih/8tbvrdKWxSz/ofuZGX
V2B21kFIuH9omFsSrYSNzsSIHs11ywCLgW5+Sb1YDYmq6XV0SQ/rKwRZni7mFsuerR5PAjbU3mflDB36V
VPjd40RlrAO9zCrdqepdNZ3+jKfIbc4IC6aNA2ukJ9SuRfsxE60w20G7K5ML1NDnHs4BntQbX1U195hj+sB
lcTDCz7sVJ9d1F36CLmK/+71IpaCmdPZmX6338pOQ4boljA3Tb4Mjnl99yRUhOiVSqJaLE2agVVsVSgV9D
gLYTjbcAZgCFe7KYUOj6Njz5nRkDfnbK0AYdvo5HEwtYfqfUJzxNU= suren@SurendraVarma

```
suren@SurendraVarma MINGW64 ~/OneDrive/Desktop/hiring app/hiring-app (main)
$ ssh-keygen
Generating public/private rsa key pair.
Enter file in which to save the key (/c/Users/suren/.ssh/id_rsa):
Enter passphrase (empty for no passphrase):
Enter same passphrase again:
Your identification has been saved in /c/Users/suren/.ssh/id_rsa
Your public key has been saved in /c/Users/suren/.ssh/id_rsa.pub
The key fingerprint is:
SHA256:hGt9ZUJRDR2/R71MlA1gysuBOGG3rBvEZpuw3YrSbMM suren@SurendraVarma
The key's randomart image is:
+---[RSA 3072]----+
|      ..o.+o==o+o|
|       =.* +  +oo|
|      o B B o  .+|
|       @ * *  o.o|
|      + S =    +.|
|     = . =      .|
|    . E o        |
|     o .         |
|                 |
+----[SHA256]-----+

suren@SurendraVarma MINGW64 ~/OneDrive/Desktop/hiring app/hiring-app (main)
$ cat^C

suren@SurendraVarma MINGW64 ~/OneDrive/Desktop/hiring app/hiring-app (main)
$ cat /c/Users/suren/.ssh/id_rsa.pub
ssh-rsa AAAAB3NzaC1yc2EAAAADAQABAAABgQCw7KrzCFbmQctVC0c173Qb89FlSqNCV3N2TSaSkIFW
X4ZQUXQ8U9Kb7vO/o48vuG5Km6xkC+J+gu/jOZhcazULAI7hzUSKS3dne7BSfE3Avz542Sc3FZXwpedF
Ff8Eap3O5vGl04AV1dT4//CpQZCZi/BinqApEmDejxztHTEX0Mhi+3M0J7iJofceKAYJXyfih/8tbvrd
KWxSz/ofuZGXV2B21kFIuH9omFsSrYSNzsSIHs11ywCLgW5+Sb1YDYmq6XV0SQ/rKwRZni7mFsuerR5P
AjbU3mflDB36VVPjd40RlrAO9zCrdqepdNZ3+jKfIbc4IC6aNA2ukJ9SuRfsxE60w20G7K5ML1NDnHs4
BntQbX1U195hj+sBlcTDCz7sVJ9d1F36CLmK/+71IpaCmdPZmX6338pOQ4boljA3Tb4Mjnl99yRUhOiV
SqJaLE2agVVsVSgV9DgLYTjbcAZgCFe7KYUOj6Njz5nRkDfnbK0AYdvo5HEwtYfqfUJzxNU= suren@S
urendraVarma
```



I used git push -u origin main to upload my local commits to the GitHub repository.

The origin represents my remote GitHub repository, and main represents the branch I want to push.

I used git push to upload my subsequent local commits to GitHub.

```
debug2: channel 0: output open -> drain
debug2: channel 0: obuf empty
debug2: chan_shutdown_write: channel 0: (i3 o1 sock -1 wfd 6 efd 7 [write])
debug2: channel 0: output drain -> closed
debug3: receive packet: type 97
debug2: channel 0: rcvd close
debug3: channel 0: will not send data after close
debug2: channel 0: almost dead
debug2: channel 0: gc: notify user
debug2: channel 0: gc: user detached
debug2: channel 0: send close
debug3: send packet: type 97
debug2: channel 0: is dead
debug2: channel 0: garbage collecting
debug1: channel 0: free: client-session, nchannels 1
debug3: channel 0: status: The following connections are open:
  #0 client-session (t4 r43 i3/0 o3/0 e[write]/0 fd -1/-1/7 sock -1 cc -1 io 0x00/0x00)

debug3: send packet: type 1
Transferred: sent 4480, received 4264 bytes, in 0.9 seconds
Bytes per second: sent 4853.8, received 4619.8
debug1: Exit status 0
Everything up-to-date
branch 'main' set up to track 'origin/main'.

suren@SurendraVarma MINGW64 ~/OneDrive/Desktop/hiring app/hiring-app (main)
$ git status
On branch main
Your branch is up to date with 'origin/main'.

nothing to commit, working tree clean

suren@SurendraVarma MINGW64 ~/OneDrive/Desktop/hiring app/hiring-app (main)
$
```

I used git pull to download the latest changes from the remote GitHub repository and integrate those changes into my current local branch.

14. Fetch changes

git fetch

git fetch origin

```
suren@SurendraVarma MINGW64 ~/OneDrive/Desktop/hiring app/hiring-app (main)
$ git status
On branch main
Your branch is up to date with 'origin/main'.

nothing to commit, working tree clean

suren@SurendraVarma MINGW64 ~/OneDrive/Desktop/hiring app/hiring-app (main)
$ git fetch

suren@SurendraVarma MINGW64 ~/OneDrive/Desktop/hiring app/hiring-app (main)
$ git fetch origin

suren@SurendraVarma MINGW64 ~/OneDrive/Desktop/hiring app/hiring-app (main)
$
```

I used git fetch to download information about changes from the remote

repository without automatically merging those changes into my current branch.

Easy difference:

- git fetch → downloads changes but does not merge.
- git pull → downloads changes and integrates them.

15. Clone a repository

git clone git@github.com:YOUR_USERNAME/hiring-app.git

cd hiring-app

```
suren@SurendraVarma MINGW64 ~/OneDrive/Desktop/hiring app/hiring-app (main)
$ git clone git@github.com:https://github.com/varmanadavapalli-art/hiring-apps.git
Cloning into 'hiring-apps'...
fatal: protocol 'git@github.com:https' is not supported

suren@SurendraVarma MINGW64 ~/OneDrive/Desktop/hiring app/hiring-app (main)
$ git clone https://github.com/varmanadavapalli-art/hiring-apps.git
Cloning into 'hiring-apps'...
remote: Enumerating objects: 3, done.
remote: Counting objects: 100% (3/3), done.
remote: Compressing objects: 100% (2/2), done.
remote: Total 3 (delta 0), reused 3 (delta 0), pack-reused 0 (from 0)
Receiving objects: 100% (3/3), done.

suren@SurendraVarma MINGW64 ~/OneDrive/Desktop/hiring app/hiring-app (main)
$
```

```
suren@SurendraVarma MINGW64 ~/OneDrive/Desktop/hiring app/hiring-app (main)
$ git clone https://github.com/varmanadavapalli-art/hiring-apps.git
Cloning into 'hiring-apps'...
remote: Enumerating objects: 3, done.
remote: Counting objects: 100% (3/3), done.
remote: Compressing objects: 100% (2/2), done.
remote: Total 3 (delta 0), reused 3 (delta 0), pack-reused 0 (from 0)
Receiving objects: 100% (3/3), done.

suren@SurendraVarma MINGW64 ~/OneDrive/Desktop/hiring app/hiring-app (main)
$ cd hiring-app
bash: cd: hiring-app: No such file or directory

suren@SurendraVarma MINGW64 ~/OneDrive/Desktop/hiring app/hiring-app (main)
$ cd hiring-apps

suren@SurendraVarma MINGW64 ~/OneDrive/Desktop/hiring app/hiring-app/hiring-apps (main)
$
```

I used git clone when I wanted to get an existing GitHub repository onto my local

machine. It downloads the repository, including its files and Git history.

16. Unstage a file

git reset F

```
suren@SurendraVarma MINGW64 ~/OneDrive/Desktop/hiring app/hiring-app (main)
$ cd hiring-apps

suren@SurendraVarma MINGW64 ~/OneDrive/Desktop/hiring app/hiring-app/hiring-apps (main)
$ git reset F
fatal: ambiguous argument 'F': unknown revision or path not in the working tree.
Use '--' to separate paths from revisions, like this:
'git <command> [<revision>...] -- [<file>...]'

suren@SurendraVarma MINGW64 ~/OneDrive/Desktop/hiring app/hiring-app/hiring-apps (main)
$ touch F

suren@SurendraVarma MINGW64 ~/OneDrive/Desktop/hiring app/hiring-app/hiring-apps (main)
$ git add A

suren@SurendraVarma MINGW64 ~/OneDrive/Desktop/hiring app/hiring-app/hiring-apps (main)
$ ls
A  B  C  F

suren@SurendraVarma MINGW64 ~/OneDrive/Desktop/hiring app/hiring-app/hiring-apps (main)
$ git status
On branch main
Your branch is up to date with 'origin/main'.

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        F

nothing added to commit but untracked files present (use "git add" to track)

suren@SurendraVarma MINGW64 ~/OneDrive/Desktop/hiring app/hiring-app/hiring-apps (main)
$
```

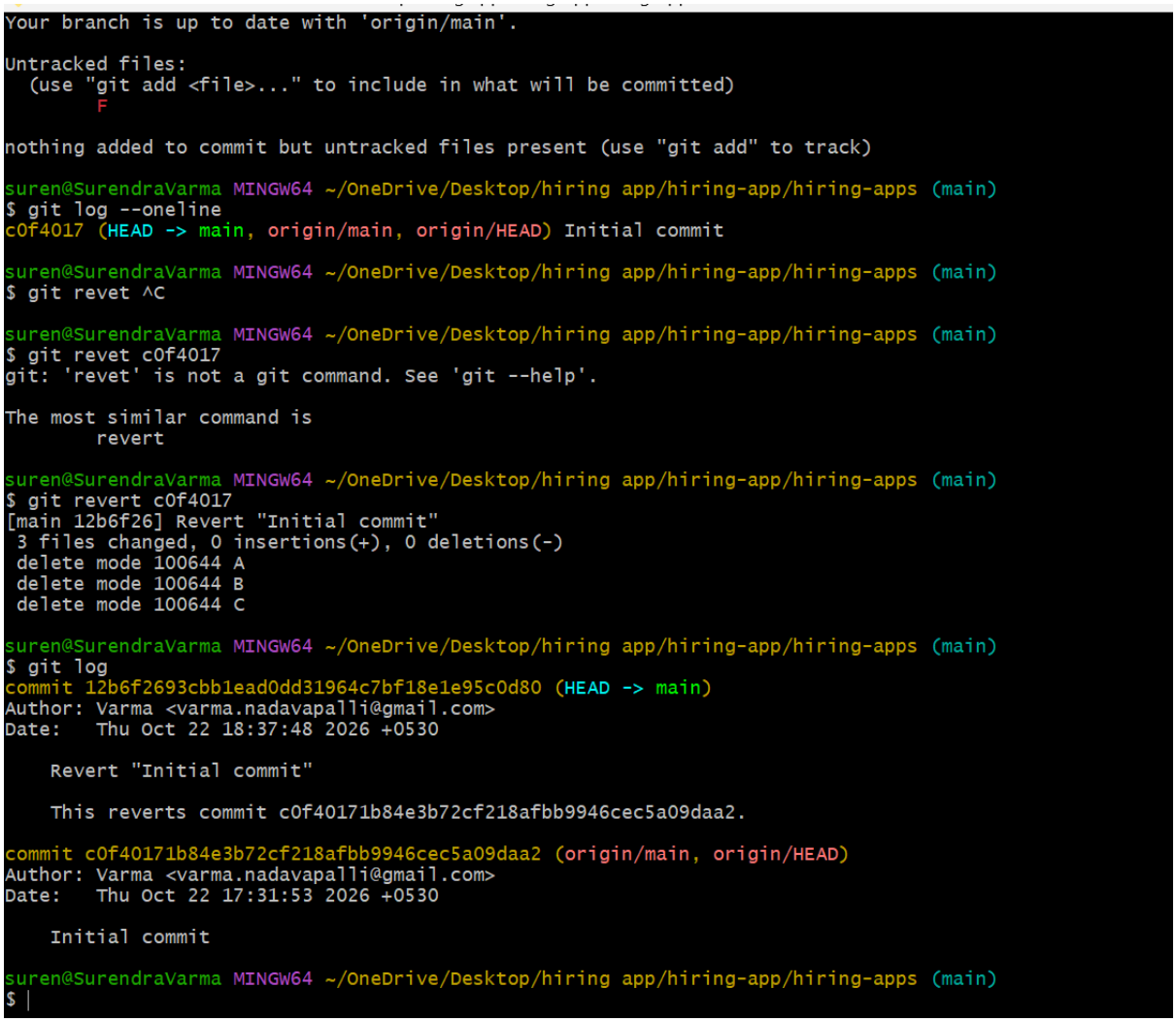
I used git reset F when I wanted to remove file F from the staging area

without deleting the file from my working directory.

17. Revert a commit

git log --oneline

git revert <commit-id>



I first used git log --oneline to find the commit ID. Then I used git revert <commit-id> to undo the changes introduced by that commit. Git creates a new commit to reverse the previous changes